
DEMKIT

INSTALLATION GUIDE

Guide version 1.0.1

26 November 2018

Gerwin Hoogsteen

Contents

1. Introduction.....	3
1.1 Changelog.....	3
2. Resources and literature	4
2.1 Contact	4
2.2 Code.....	4
2.3 Literature	4
2.4 Third party documentation	5
2.5 Tutorials.....	5
3. Requirements	6
3.1 Software	6
4. Installing DEMKit	8
4.1 Retrieving DEMKit	8
4.1.1 Installing Git.....	8
4.1.2 Generating an SSH-key	9
4.1.3 Pulling DEMKit	9
4.1.4 Editing the DEMKit config.....	11
4.2 Installing Python 3 and packages	11
4.2.1 Installing Python	12
4.2.2 Installing Python packages	12
4.3 Installing InfluxDB and Grafana	13
4.3.1 Installing InfluxDB.....	13
4.3.2 Installing Grafana	14
4.3.3 Setting up Grafana.....	15
4.4 Setting up PyCharm	16
4.4. Loading DEMKit as a project.....	16
5. Running DEMKit.....	20
5.1 Running a simulation	20
5.2 Visualizing data with Grafana.....	21
5.3 Generating ALPG data	23
6. Coding Guidelines.....	25
6.1 DEMKit framework	25
6.2 Scenarios	25

1. Introduction

This document serves as an installation guide of the DEMKit software. Furthermore, it provides some pointers to run the provided demo and how to extend this demo with data generated with the open-source artificial load profile generator.

The first chapter gives an overview of DEMKit resources (code and literature). Followed by a chapter describing the required software and packages to run DEMKit, such that advanced users know all requirements for the operating systems Windows, MacOS and Linux, followed by an extensive step-by-step instruction for Windows based systems. Instructions on how to run a simulation and interpret the data are provided in the fourth chapter. The last chapter of this guide describes coding guidelines.

1.1 Changelog

Date	Version	Remarks
26-11-2018	1.0.1	Typo's
07-11-2018	1.0	Initial guide

2. Resources and literature

2.1 Contact

The maintainers of DEMKit can be reached through e-mail for general contact about support, questions, features, etc. The email address is: demgroup-eemcs@utwente.nl

In case there is no prompt response, please contact the main DEMKit maintainer:

Gerwin Hoogsteen

email: g.hoogsteen@utwente.nl

2.2 Code

The code is hosted in 2 git repositories at the University of Twente, the Netherlands. Access to the repository is managed by the DEMKit maintainers.

University of Twente Version Control System (VCS) location:

<https://vcs.utwente.nl/>

Specific resources:

DEMKit core:	https://vcs.utwente.nl/source/DEMKit/
DEMKit example:	https://vcs.utwente.nl/source/DEMkit_example/
Workboard:	https://vcs.utwente.nl/project/view/87/
Project website:	https://www.utwente.nl/en/eemcs/energy/

Often, the VCS system does not display a login-screen. If this is the case, click on the symbol with the door in the top right corner. Log in using these credentials:

Employees of University of Twente:

Login using LDAP (lower block). Use your email username (e.g. hoogsteeng) and password.

Students of the University of Twente:

Login using LDAP (lower block). Use your student number (e.g. s1234567) and password.

Other:

Use the upper block. Register for an account and log in using these credentials. Contact us to set the proper permissions to access the DEMKit software.

2.3 Literature

For more background on the architecture of DEMKit and the implemented components and optimization algorithms, we refer to our publications. A comprehensive list of all our publications can be found on our website: <https://www.utwente.nl/en/eemcs/energy/publications/>

Specific information about the architecture of DEMKit and optimization methods are provided in: G. Hoogsteen, "A cyber-physical systems perspective on decentralized energy management", PhD dissertation, 2017, University of Twente, the Netherlands.

Available at: <https://doi.org/10.3990/1.9789036544320>

Specific information about coordination and device level optimizations are provided in:
T. van der Klauw, “Decentralized Energy Management with Profile Steering: Resource Allocation Problems in Energy Management”, PhD dissertation, 2017, University of Twente, the Netherlands.

Available at: <https://doi.org/10.3990/1.9789036543019>

2.4 Third party documentation

DEMKit uses external tools and software, as also described in the requirements. The following links provide links to documentation of these tools:

- ALPG: <https://github.com/GENETX/alpg>
- Grafana: <http://docs.grafana.org/>
- InfluxDB: <https://docs.influxdata.com/influxdb/v1.6/>
- Python:
 - Astral: <https://astral.readthedocs.io/en/stable/index.html>
 - Eve: <http://docs.python-eve.org/en/latest/>
 - Numpy: <https://docs.scipy.org/doc/>
 - Pytz: <https://readthedocs.org/projects/pytz/>
 - Requests: <http://docs.python-requests.org/en/master/>
 - ZMQ: <https://pyzmq.readthedocs.io/en/latest/api/zmq.html>

For project management, the following documentation is available:

- Git: <https://git-scm.com/doc>
- Phabricator: <https://secure.phabricator.com/book/phabricator/>

2.5 Tutorials

There is no specific tutorial on using DEMKit yet, please refer to the DEMKit example to get familiar with the tool. However, tutorials on Python may be useful to learn how to adapt and create your own scripts as part of the configuration files. These interactive tutorials should provide a good starting point:

<https://www.learnpython.org/>

<https://www.codecademy.com/learn/learn-python>

<https://www.datacamp.com/courses/intro-to-python-for-data-science>

For Grafana, the getting started guide can be useful:

http://docs.grafana.org/guides/getting_started/

3. Requirements

The DEMKit software makes use of third-party tools and software to simulate, interact, store and visualize data. This chapter gives an overview of the required tools and packages which are generally available for all major operating systems and also for Linux powered embedded systems using ARM processors. Advanced users should be able to install the software given this list and common knowledge of tools such as python, pip and Git. An extensive guide for Windows users is provided in the next chapter, which also presents the information given in this chapter. Instructions to run a simulation are given in Chapter 5.

3.1 Software

The following software, often running as services, is required for DEMKit and the interfacing with the version control system. A tested version number is provided, however we encourage everybody to test/run the most recent stable versions of these tool. Links to documentation, if needed, is provided in the previous Chapter.

- Git:
 - Download: <https://git-scm.com/downloads>
 - Tested version: 2.16.4
 - Notes: -
- Python 3:
 - Download: <https://www.python.org/downloads/>
 - Tested version: 3.6.5
 - Notes: Make sure you use Python 3.x instead of 2.x. On some operating systems this is achieved by running using “python3” instead of just “python”. The same applies to installing packages though pip (“pip3”). In Windows, do not forget to tick the add python to PATH-box during installation!
- InfluxDB:
 - Download: <https://portal.influxdata.com/downloads>
 - Tested version: 1.5.4
 - Notes: On Windows, do not forget to start (or make it autostart) before running DEMKit
- Grafana:
 - Download: <https://grafana.com/grafana/download>
 - Tested version: 5.1.3
 - Notes: On Windows, do not forget to start (or make it autostart) before running DEMKit
- ALPG (recommended)
 - Download: <https://github.com/GENETX/alpg>
 - Tested version: 1.3
 - Notes: -

In addition, for development of DEMKit and scenarios, we recommend the following tools, however you are free to use the tools you prefer:

- Notepad++ (Windows only)
 - Download: <https://notepad-plus-plus.org/download/v7.5.9.html>
 - Tested version: 7.5.9
 - Notes: This is not the normal notepad and provides features like code highlighting. Ideal for quick changes to e.g. config files.
 -
- PyCharm IDE
 - Download: <https://www.jetbrains.com/pycharm/download/>
 - Tested version: 2018.1.4
 - Notes: An academic license can be obtained free of charge for employees of most academic institutions. This lets you use the Professional version. Otherwise, the free Community version is likely to provide enough features.

In addition, the following python modules are, which can be installed using “pip”, e.g. using the following command in a command prompt:

```
python -m pip install <package>
```

Note to check if the right version of python (2.x or 3.) is used. Some operating systems require admin privileges, e.g. “sudo python ...” or an “elevated prompt” (Windows) by running the console as admin (right-click on cmd.exe, “run as administrator”).

Required Packages:

- astral
- numpy
- requests
- pytz
- zmq

Commandline:

```
python -m pip install astral numpy requests pytz zmq
```

Optional Packages:

- eve
- pyserial

Commandline:

```
python -m pip install eve pyserial zmq
```

An up-to-date list is always provided in the README.md in the repository. The Readme also provides simple tips-and-tricks on the installation and configuration. The readme file is accessible here:

<https://vcs.utwente.nl/source/DEMKit/browse/master/README.md>

4. Installing DEMKit

This is a step-by-step guide to install DEMKit. The guide is mostly focussed on Windows users, but gives pointers for those running UNIX-based (Linux, MacOS) systems.

4.1 Retrieving DEMKit

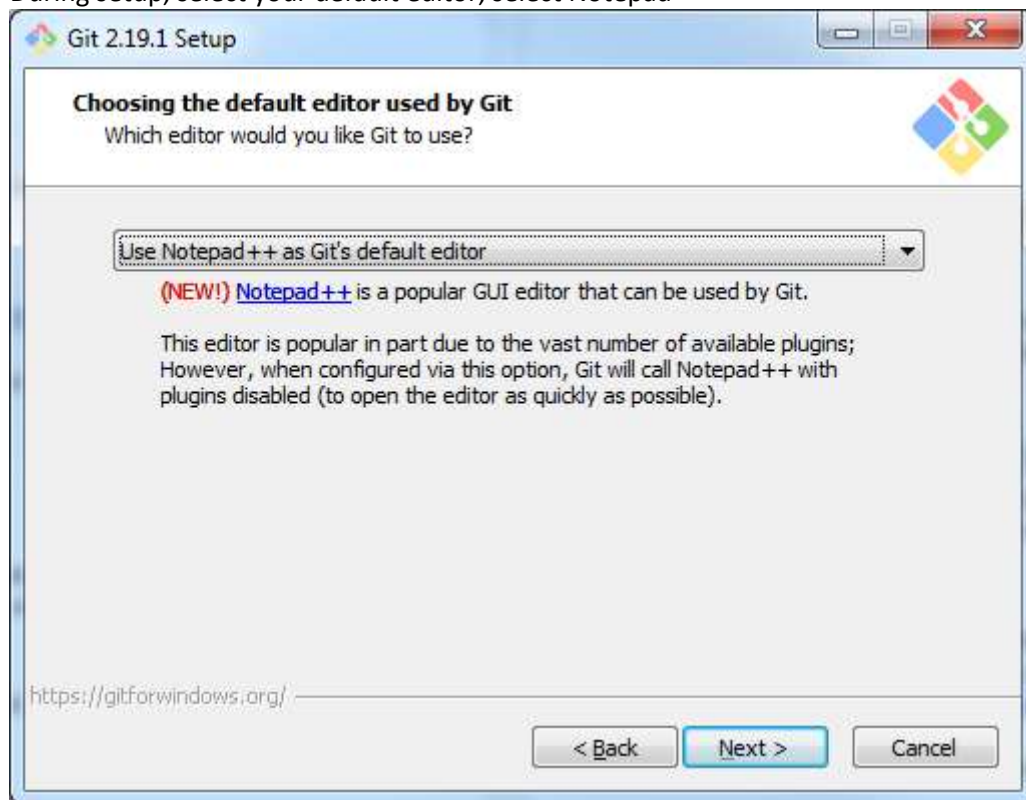
Git is required to download DEMKit. Most Linux distributions have Git installed by default as a commandline tool and can skip to 4.1.2.

4.1.1 Installing Git

WINDOWS

For Windows, we advice to install Notepad++ first from <https://notepad-plus-plus.org/download/v7.5.9.html>. Then download and install Git. Executables are found here: <https://git-scm.com/downloads> .

During setup, select your default editor, select Notepad++



The other default options ticked in the installation process are fine. Git does not install bloatware, so it is safe to continue by clicking next.

LINUX

Linux users may prefer using their package manager instead.

MAC

No instructions available.

4.1.2 Generating an SSH-key

For security reasons, an SSH key is required to download the DEMKit code from the version control system. This key needs to be added on the version control system website. First, we obtain an SSH-key based on the operating system. Then, uploading this key is similar for all operating systems.

WINDOWS

- Start the tool called Git GUI
- Click “Help” and then “Show SSH Key”
- If there is no SSH Key, create one by clicking “Generate Key”. It is advised not to set a password by just clicking “OK”.
- Click “Copy to clipboard”
- Keep Git GUI open, but close the “Your OpenSSH Public Key” screen.

LINUX

- Open a terminal
- First check if you already have a key using this command:
`cat ~/.ssh/id_rsa.pub`
If it displays a key, then you can skip the next step.
- Generate a new key by:
`ssh-keygen -t rsa`
Follow the instructions. It is most convenient to skip the setting a passphrase part by simply pressing enter.
- After generation, copy the key by:
`cat ~/.ssh/id_rsa.pub`
Select and copy the output, which looks like:
`ssh-rsa AAAA ... 4HQ1 user@systemname`

MACOS

See Linux.

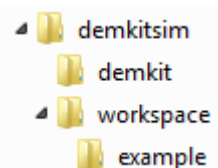
Uploading the key

Assuming the key is copied correctly, go to <https://vcs.utwente.nl/settings> and follow these instructions:

- Click “SSH Public Keys” on the left-side
- Click “SSH Key Actions” and then “Upload Public Key”
- Enter a name, preferably one to identify your computer easily
Paste the aforementioned SSH Key (e.g. `ssh-rsa AAAA ... 4HQ1 user@systemname`) into the lower text box.
- Click “Upload Public Key”
- You’re done her

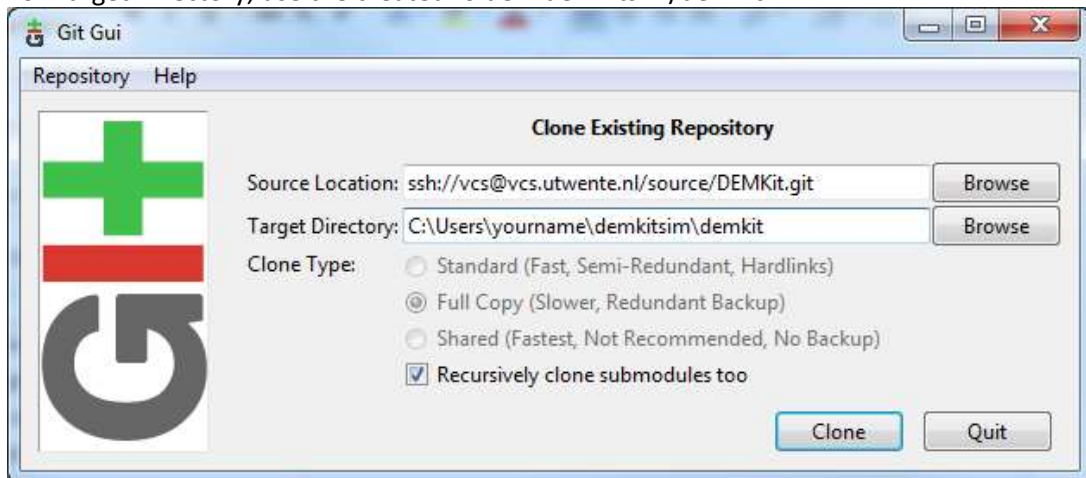
4.1.3 Pulling DEMKit

Now we are ready to download the code of DEMKit and an example scenario. For this setup, we assume that you have created a folder called “**demkitsim**” somewhere on your PC, e.g. in your “home” folder (Windows: `C:\users\yourname\demkitsim\` or UNIX: `~/demkitsim/`). We will refer to “demkitsim” as the folder in which we install all code and tools. At the end of this section you should end up with the folder structure depicted on the right.

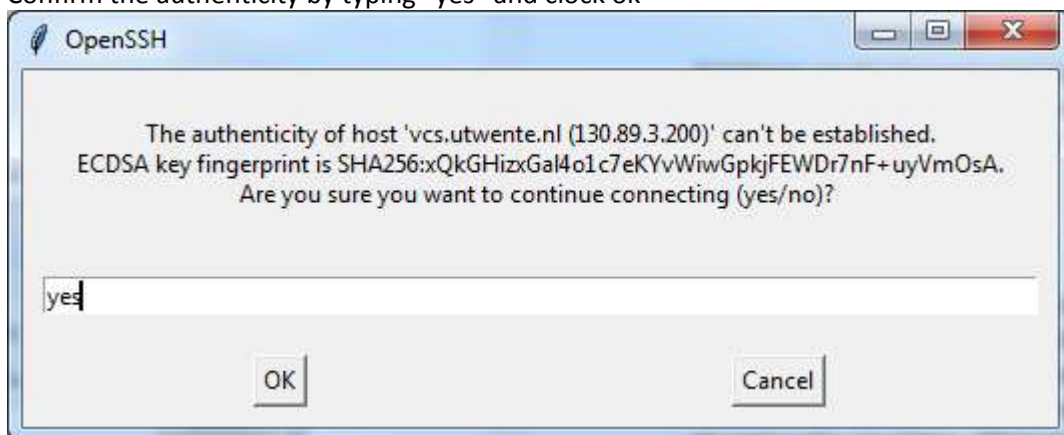


WINDOWS

- First, let us download the simulator software
- Click on “Clone existing repository”
- Use the following Source Location: `ssh://vcs@vcs.utwente.nl/source/DEMKit.git`
For Target Directory, use the created folder “demkitsim/demkit”.



- Click “Clone”
- Confirm the authenticity by typing “yes” and click ok

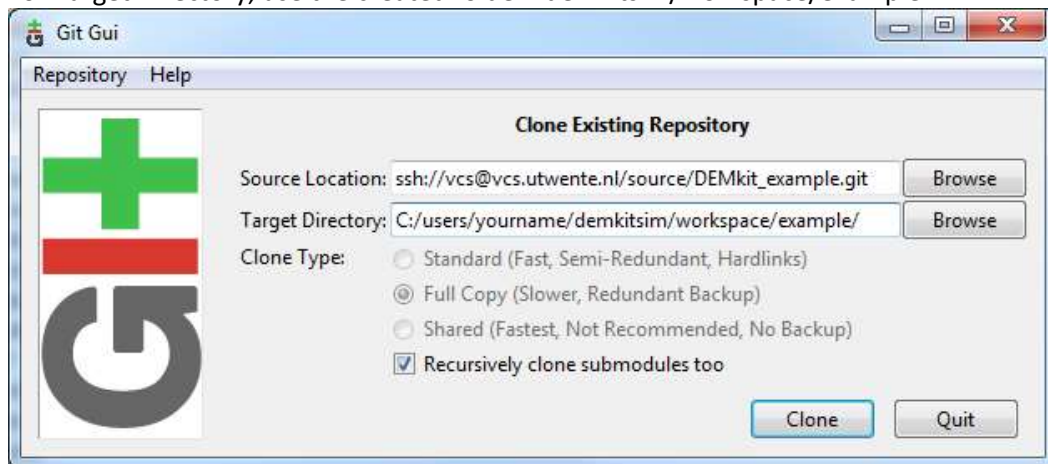


- Wait till Git finishes downloading the code, then it is safe to close



- To download the example: open another Git GUI windows

- Click on “Clone existing repository”
- Use the following Source Location: `ssh://vcs@vcs.utwente.nl/source/DEMkit_example.git`
For Target Directory, use the created folder “demkitsim/workspace/example”.



- Click “Clone” and again, wait till Git completes cloning

LINUX

- Open a terminal
- Navigate to the created “demkitsim” folder, e.g.:
`cd ~/demkitsim`
- Now clone the demkit code
`git clone ssh://vcs@vcs.utwente.nl/source/DEMKit.git`
- Now clone the demkit example
`mkdir workspace`
`cd workspace`
`git clone ssh://vcs@vcs.utwente.nl/source/DEMKit_example.git`
`cd ..`

MACOS

See Linux.

4.1.4 Editing the DEMKit config

After downloading the DEMKit software, it is required to configure some environment variables in the DEMKit config:

- Open the file “demkitsim/demkit/conf/usrconf.py.misc” with a text editor, e.g. notepad++
- Change the following settings to reflect your setup. The componentPath and workspacePath are essential. Based on the example, the following values should be used:

```
componentPath = 'C:/users/yourname/demkitsim/demkit/components/'
workspacePath = 'C:/users/yourname/demkitsim/workspace/'
```

The other values can be kept as they are. **Make sure you end with a trailing slash**
For Windows users, do use “/” instead of “\”, or use an escaped backslash “\\”

- Save the file as “usrconf.py”! **Make sure the “.misc” is removed!**

4.2 Installing Python 3 and packages

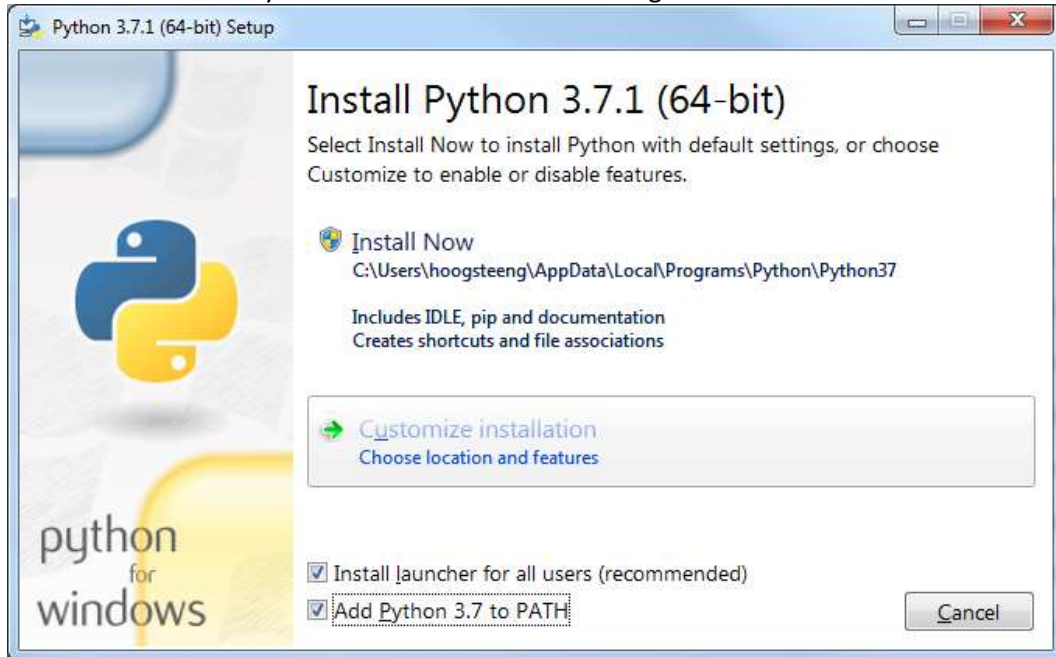
DEMKit is written in the Python 3 language, hence Python 3 is required, together with some packages. This section first explains how to install Python 3 and then the required packages.

4.2.1 Installing Python

If you already have Python installed, make sure it is version 3!

WINDOWS

- Download the Python 3 installer from: <https://www.python.org/downloads/release/>
- Open the installer.
- Check the box “Add Python 3.x to PATH” as in the image below



- Then select “Install Now”
- Wait till the installer finishes.

LINUX

Usually, Python 3 is already installed. You can check this by opening a terminal and type “python” or “python3”. In some distributions, “python” may start version 2.x, whereas “python3” starts python 3.x. If Python 3 is not yet installed, check your package manager.

MACOS

- Install the Command Line Tools for Xcode (required for python) by typing:
`xcode-select --install`
in a Terminal window.
- Install Homebrew from <https://brew.sh>. Homebrew is a package manager for Mac OS X, just as you might have for a Linux system.
- Install python3 using Homebrew, type:
`brew install python3`
in a Terminal window.

4.2.2 Installing Python packages

After installing Python, additional libraries are required.

WINDOWS

- Open a commandline as administrator:
 - Start, search for “cmd.exe”
 - Right-click cmd.exe, choose “run as administrator”
- For the required packages, type:
`pip install astral numpy requests pytz zmq`
- For additional (not required) packages, type:
`pip install eve pyserial`
- Close the console

Note, optionally you may need to install a part of the Visual Studio software. Check the errors that the terminal provides

LINUX

- Open a terminal:
- For the required packages, type:
`sudo pip3 install astral numpy requests pytz zmq`
- For additional (not required) packages, type:
`sudo pip3 install eve pyserial`
- Close the console

Note, pip3 may not exist on your distribution. Instead, try to replace “pip3” by “pip”, “python3 –m pip” or “python –m pip”. Make sure the packages are installed for Python 3.x. If no installation of pip exists, please install pip through your package manager.

MACOS

See Linux.

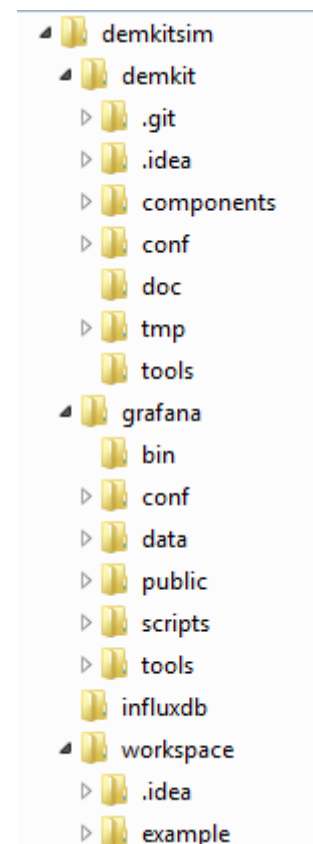
4.3 Installing InfluxDB and Grafana

DEMKit uses an external database (InfluxDB) and web-based data viewer (Grafana) to store and present results of simulations. For Windows, the final folder structure should look like the image on the right.

4.3.1 Installing InfluxDB

WINDOWS

- Download InfluxDB from:
<https://portal.influxdata.com/downloads>
- Extract the contents to “demkitsim”
- Rename the influxdb-x.x.x folder to simply “influx”
- Navigate into the “influx” folder
- Start influxd.exe
Click “Run” if a warning appears



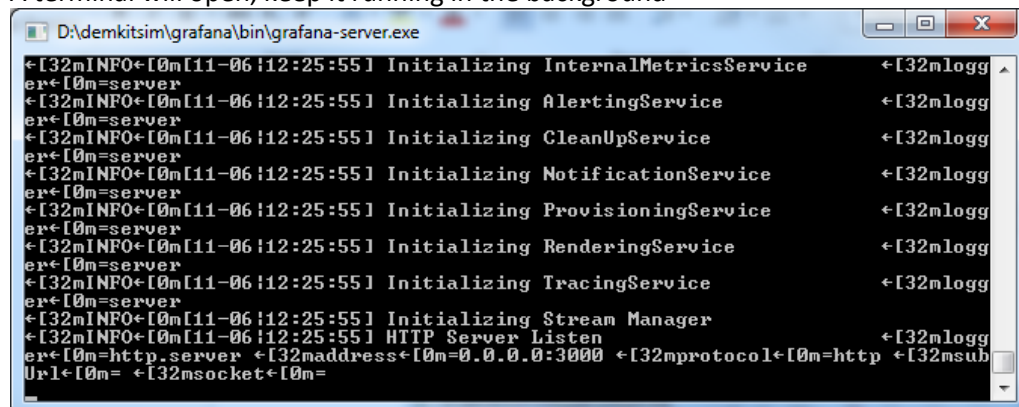
```

D:\demkitsim\influxdb\influxd.exe

88888888      .d888 888      888888888b. 8888888b.
888           d88P" 888      888      "Y88b 888      "88b
888           888   888      888      888 888      .88P
888 888888b. 8888888 888 888 888 888 888 888 88888888K.
888 888 "88b 888 888 888 888 888 888 888 888 "Y88b
888 888 888 888 888 888 888 888 X88K 888 888 888 888
888 888 888 888 888 888 Y88b 888 .d8""8b. 888 .d88P 888 .d88P
88888888 888 888 888 888 "Y88888 888 888 88888888P" 88888888P"

2018-11-06T11:07:03.969704Z      info    InfluxDB starting    {"log_id": "
6sdG000", "version": "1.6.4", "branch": "1.6.", "commit": "a2ba6e7654fb078f8a
dd1f8d935af38161bd"}
2018-11-06T11:07:03.969704Z      info    Go runtime            {"log_id": "0BbB6sdG

```



LINUX

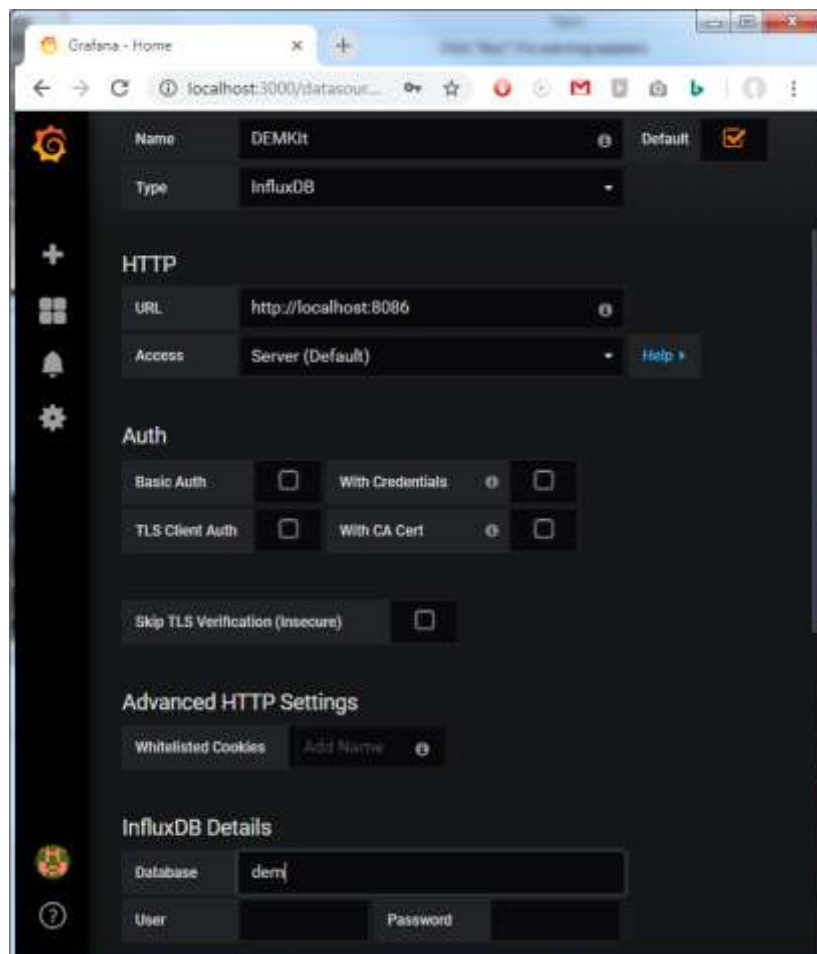
- Install grafana from your package manager or refer to instructions on <https://grafana.com/grafana/download>
- It is recommended to start Grafana on boot, e.g. through systemctl
- Alternatively, it can be started from the commandline:
grafana-server

MACOS

- Install Grafana from the terminal using Brew:
brew install grafana
brew services start grafana

4.3.3 Setting up Grafana

- Open a browser and browse to the Grafana page located at <http://localhost:3000>
- Log in. Default username is “admin” and password “admin”.
- Set a new password if desired or skip.
- Go to the settings by hovering over the cogwheel on the left and clicking “Data Sources”
- Then select “Add Datasource”
- Fill in the following details:
 - Name: “DEMKit”
 - Default: Yes (tick)
 - Type: InfluxDB
 - URL: <http://localhost:8086> (if it is grey, you still need to fill it!)
 - Database: “dem”
 - The rest can be kept blank
- Click “Save and Test”
- Most likely, you will get an error as the database does not exist. This is not a problem as it will be created during the first simulation. If you have problems later, return to these steps and check if the source is working.



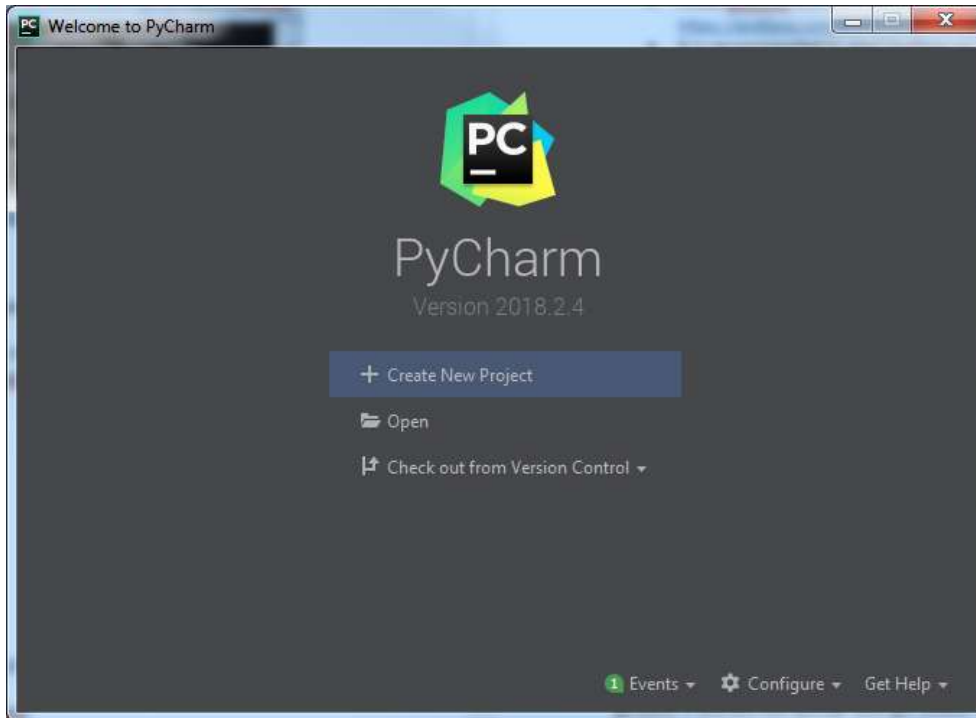
4.4 Setting up PyCharm

We recommend PyCharm as IDE for development. Installation of PyCharm should be trivial. PyCharm can be downloaded from <https://www.jetbrains.com/pycharm/download/> (Note: there is an academic license for PyCharm Professional).

The remainder of this Section details how to import the code, change some settings and how to update.

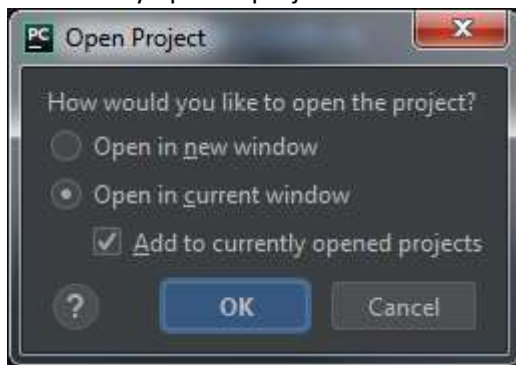
4.4. Loading DEMKit as a project

After installing and setting up PyCharm for the first time (e.g. preferences and license), a Welcome screen is greeting you:



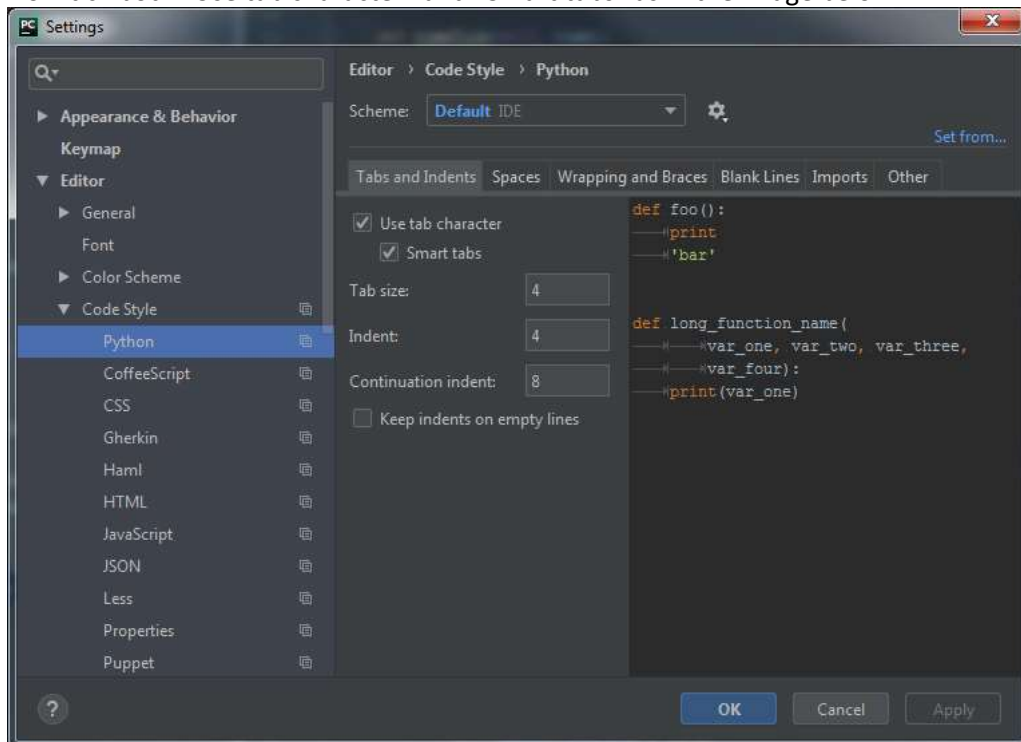
- In this screen, select Open.
- Then select the "demkitsim\demkit\"-folder
- Wait a while till it completes loading
- On the left pane, you should see the "DEMKit project, with all the source files if you collapse it.
- The next step is to also open the "workspace" directory for your scenarios
- Click "File", then "Open"
- Select the workspace directory "demkitsim\workspace\"

- If prompted how to open the project, choose “Open in current window” and also select “Add to currently opened projects”

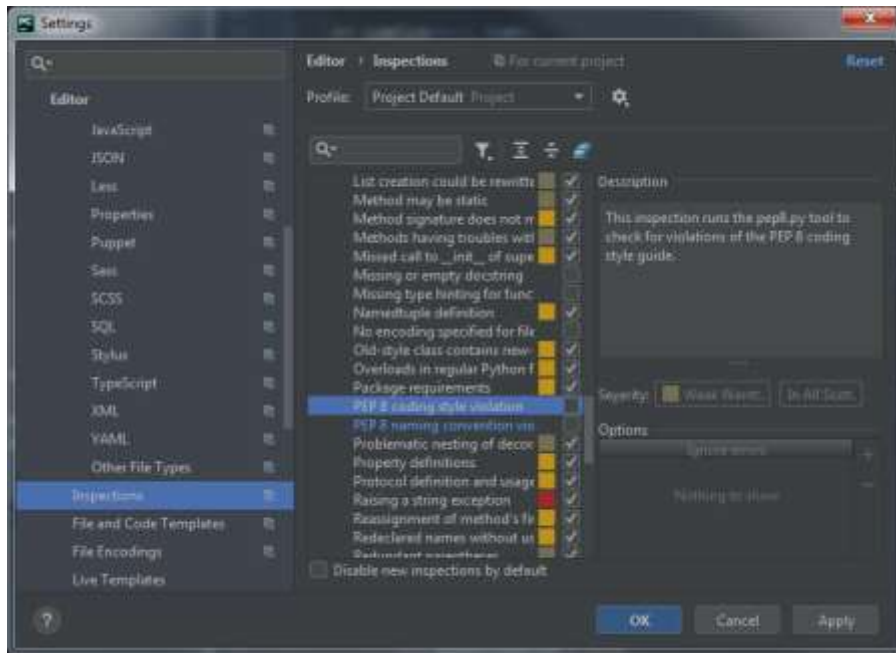


By default, PyCharm uses Python coding style guidelines, which DEMKit does not completely follow. To ease life and remove ignoring warning, the default PyCharm settings need to be adapted.

- Open “File” and then “Settings”
- Go to “Editor”, then “Coding Style”, then “Python” and select the “Tabs and Indents” tab
- Now tick both “Use tab character” and “Smart tabs” as in the image below



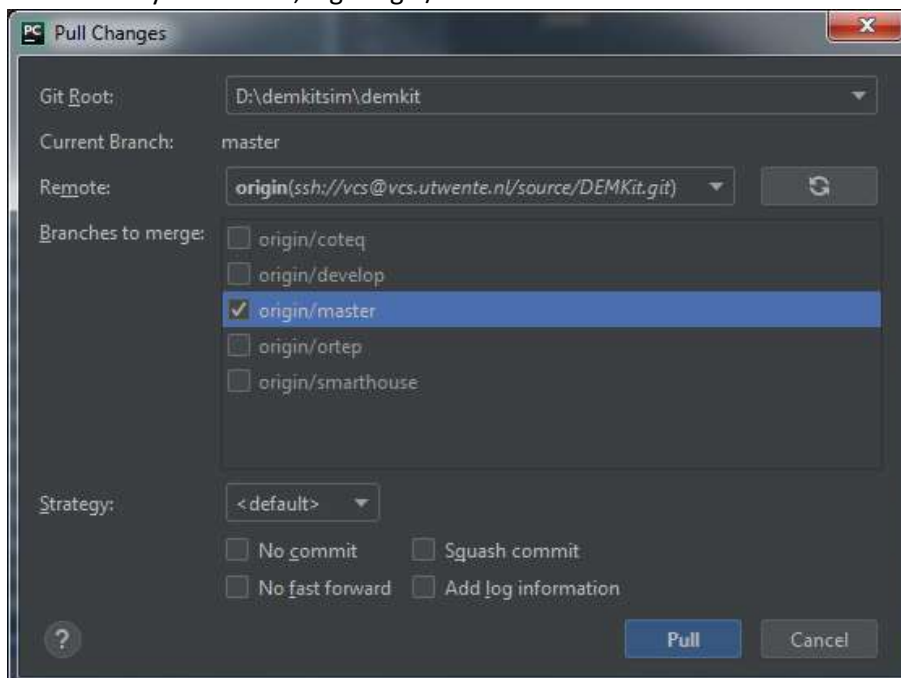
- Then go to “Editor”, then “Coding Style”, then “Inspections”.
- Select “Python”
- Now look for the following inspections and disable them:
 - PEP8 Coding style violation
 - PEP8 naming convention violation



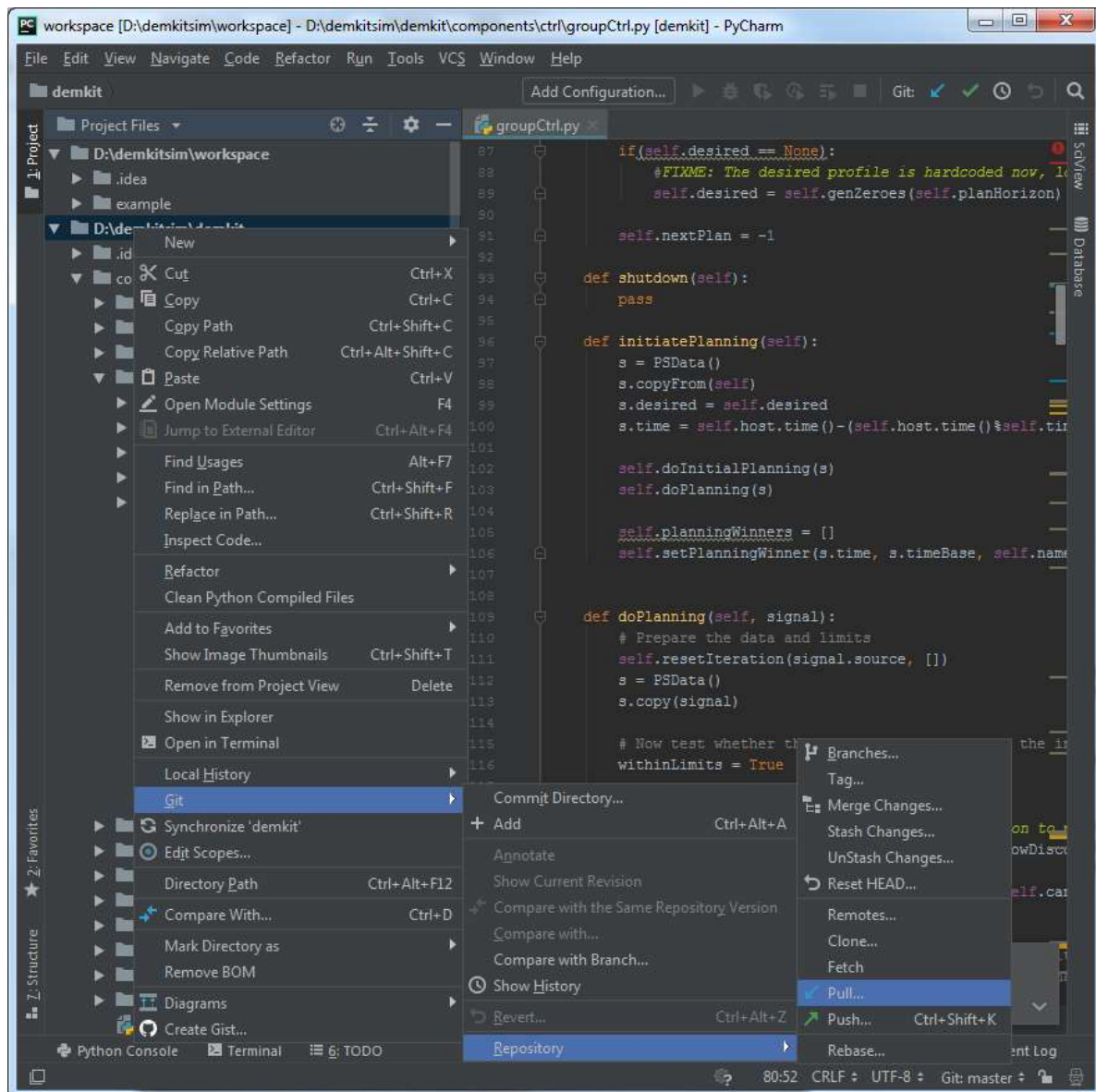
- Click “Apply” and then “OK”

The last trick with PyCharm is to easily update DEMKit and the example through Git. We only focus on getting updates, “Pulling” in Git, advanced users can also push and branch from Pycharm. Updating is done as follows:

- Right-click on the DEMKit project folder in PyCharm
- Then select “Git” , then “Repository” and then “Pull” (See picture on the nextpage)
- Then select your branch, e.g. origin/master and click “Pull”:



- In case of errors, make sure to revert your changes first (read the docs on how to do this).



5. Running DEMKit

This chapter assumes that you have correctly downloaded DEMKit and installed the required packages and software. It also assumes that your config is correctly, see Section 4.1.4.

5.1 Running a simulation

A simulation is started from a terminal. By default, an example model is provided, together with sample input data for a single house. This example can be extended by generating data for more houses (see Section 5.3) and serves as a template to create your own scenarios.

WINDOWS

First you will need to navigate to the right folder to execute a simulation

- Open up a terminal (Start, "cmd.exe")
- Navigate to the demkitsim folder, e.g.
`cd C:\Users\Yourname\demkitsim`
- Navigate to the demkit folder.
`cd demkit`

LINUX

First you will need to navigate to the right folder to execute a simulation

- Open up a terminal
- Navigate to the demkitsim folder, e.g.
`cd ~\demkitsim`
- Navigate to the demkit folder.
`cd demkit`

MACOS

See Linux.

From here, you can simulate a scenario using two paramers:

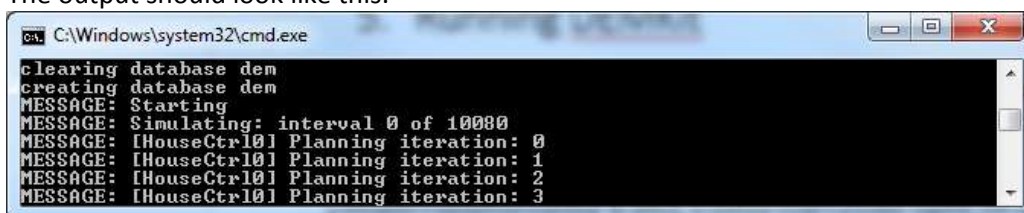
- **Model (-m):** The scenario input file, in the example *demohouse*. The model name corresponds to the filename of the scenario. Note that .py must not be included.
- **Folder (-f):** The folder where the above scenario is located. The path is relative to the configured path *workspacePath* in the *usrconf.py* file as described in Section 4.1.4.

To run the example scenario, issue the following command in the terminal (all operating systems):

```
python demkit.py -f example -m demohouse
```

The command above will load *demkitsim/workspace/example/demohouse.py*. Note that on some operating systems *python* should be replaced by *python3* to use Python 3.x.

The output should look like this:



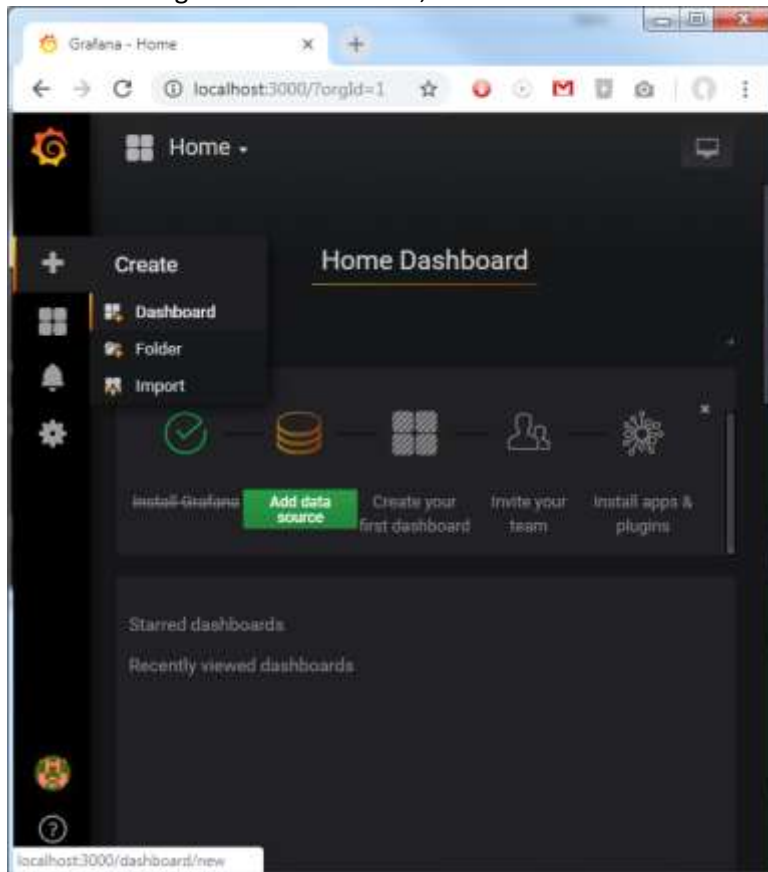
```
C:\Windows\system32\cmd.exe
clearing database dem
creating database dem
MESSAGE: Starting
MESSAGE: $imulating: interval 0 of 10000
MESSAGE: [HouseCtrl0] Planning iteration: 0
MESSAGE: [HouseCtrl0] Planning iteration: 1
MESSAGE: [HouseCtrl0] Planning iteration: 2
MESSAGE: [HouseCtrl0] Planning iteration: 3
```

5.2 Visualizing data with Grafana

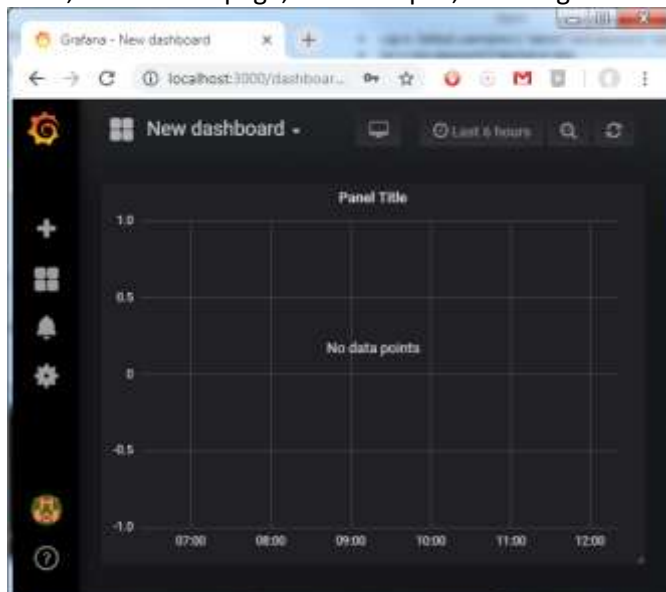
The results of a simulation can be visualized with Grafana. Querying data from the InfluxDB is made simple with this web-based tool. However, there are some tricks and pitfalls that you need to be aware of. Grafana can be accessed in a web browser at <http://localhost:3000>.

Data is visualized with a dashboard, which can be saved. A dashboard can contain multiple graphs as well. For the first time, we a new dashboard needs to be set-up:

- Click on the big “+” on the left side, and choose “Create Dashboard”



- Then, on the next page, click “Graph”, resulting in an empty graph:



- Right click on “Panel Title” and choose “Edit”
- Now you can add queries at the bottom of this page to create plots.

QUERYING

Grafana is very flexible in what data you want to visualize and how. It is therefore important to carefully construct the query to only show the data you want to see. First we explain what can be plotted, followed by the tips and tricks on HOW it should be plotted.

A query starts with selecting the data in the “FROM default <measurement>” line
DEMKit writes the results of different components to different “measurements”:

- devices (all device stats)
- controllers (control and optimization outcomes)
- flow (for load flow analysis of e.g. electricity networks)
- host (overall stats of the simulation)

Normally, this will result in data from all devices, controllers, flows or hosts. Usually, you want to target a specific type of device. This can be defined through filtering in the “WHERE”-block. Each component has a type (devtype for device type) to select all devices of the same type and a unique name to select a single component.

Regarding the types, we use abbreviations of the components described in literature:

- **meter**: MeterDevice, metering devices such as smart meters.
- **unc**: UncontrollableDevice, e.g. a base load
- **curt**: CurtailableDevice, static load/generation that can be curtailed / shed
- **buf**: BufferDevice, buffers such as electrical energy storage or heat tanks
- **conv**: Converter device, such as an heat pump
- **bufConv**: Combination of BufferDevice and ConverterDevice
- **ts**: TimeShiftableDevice, e.g. washing machines or dishwashers.
- **bts**: BufferTimeShiftable, e.g. an electric vehicle (buffer device with time limitations)

Lastly, one can select what field (value) to plot in SELECT. A long list is provided, but not all components will have data for the chosen field. The fields follow this naming convention:

Unit-type[.optional.number.of.specifiers][.c.commodity][.complex-part]

Examples:

Wh-energy-consumption.c.power

W.power.imported.c.power.real

W.power.imported.c.power.imag

C.temperature.indoor

TIPS AND TRICKS

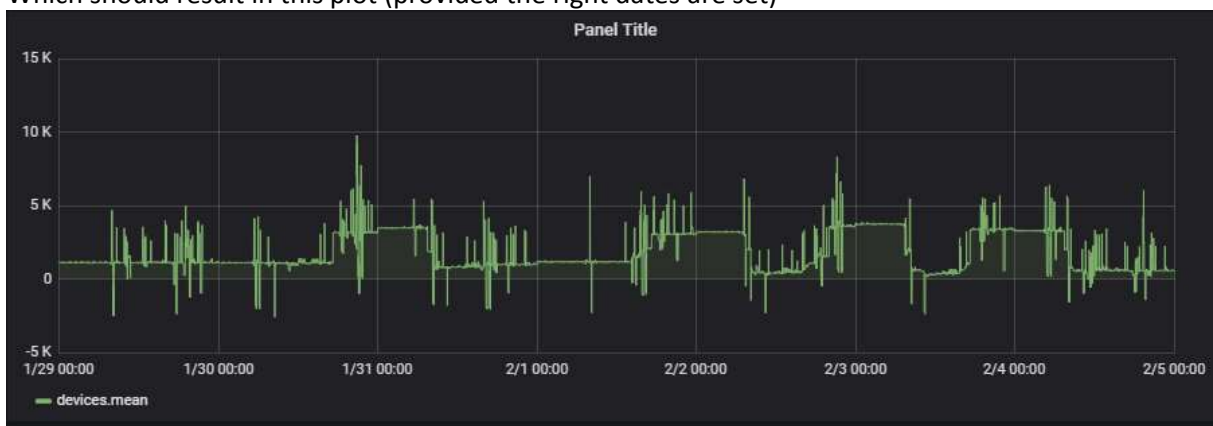
- By default a new plot only show the “Last 6 Hours” (see top right). DEMKit stores the data from the date specified in the simulation model, which by default is 1 Januari 2018. The fastest method is to display “This year” in the drop-down menu when clicking on “Last 6 hours”. Dragging can be used to zoom.
- The data plotted in Grafana is often resampled through the “GROUP BY” settings. The time herein is variable by default (“\$__interval”), which may result in unexpected outcomes. To have full control over how the data is plotted, preferably set the time grouping to 1m (m for minute) or the time interval configured in the model.

- Note that the aggregation type in SELECT (by default “mean”) also influences how data is shown. Grouping by e.g. 1 hour will result in a graph that shows the average value for all samples, 60 in this case. Even more importantly, when plotting the data of multiple components (e.g. all meters, say 100 in total), you would get the average value of all samples again, so in this case 100 components times 60 minutes = the average of 6000 values.
- Selecting a small aggregation time, but plotting a lot of data may sometimes result in no data to be shown at all at first sight. This is often due to the fact that not enough samples exist, resulting in “null” to be displayed. It is recommended to set, in “GROUP BY” the “fill” to “previous”
- Data can be aggregated when desired. E.g. to see the total, summed, power consumption of all meters, it better to select all meters in the FROM-line, and then select the “sum” “aggregation” in the “SELECT” line.
- ALIAS BY can be used to set a legend
- CSV files (e.g. for plotting in Excel or PGFPlots for Latex) can be downloaded by right-clicking the panel title and then choosing “More”, then “Export CSV”.

As an example, to plot the total energy consumption (as measured by the meter) of the demo house, the following query is the result:

▼ A	FROM	default	devices	WHERE	devtype	=	meter	AND	name	=	SM-House-ph1-0	+
	SELECT	field (W-power.real.c.POWER)		mean ()	+							
	GROUP BY	time (1m)	fill (previous)	+								
	FORMAT AS	Time series ▼										
	ALIAS BY	Naming pattern										

Which should result in this plot (provided the right dates are set)



For more information, we refer to the Grafana documentation: <http://docs.grafana.org/>

5.3 Generating ALPG data

Larger scenarios can easily be generated using the open-source Artificial Load Profile Generator (ALPG) that can be obtained at <https://github.com/GENETX/alpg>. DEMKit has build-in functionality to load such a scenario. An example is given with the example “demostreet” model, which serves as a template for your own scenarios. To generate data using the ALPG and loading the demostreet simulation, follow these steps:

WINDOWS

First you will need to navigate to the right folder to execute a simulation

- Open up a terminal (Start, "cmd.exe")
- Navigate to the demkitsim folder, e.g.
`cd C:\Users\Yourname\demkitsim`
- Navigate to the demkit example folder.
`cd workspace\example`

LINUX

First you will need to navigate to the right folder to execute a simulation

- Open up a terminal
- Navigate to the demkitsim folder, e.g.
`cd ~\demkitsim`
- Navigate to the demkit example folder.
`cd workspace\example`

MACOS

See Linux.

Now you can pull the ALPG and generate data. For all operating systems you can enter in the terminal:

```
git clone https://github.com/GENETX/alpg.git
cd alpg
python profilegenerator.py -c example -o demo -force
```

Now, and this takes a while, the data should be generated. When the ALPG is done, you can run a simulation. First navigate back to the demkit-folder:

```
cd ../../../../demkit
```

Then simulate the demostreet

```
python demkit.py -f example -m demostreet
```

For your own creations, it is best to copy the example-folder and rename it. Accordingly, change all references to "example" with the name of your own scenario. Change and alter the models as you wish.

6. Coding Guidelines

DEMKit has no strict coding guidelines, as long as things remain manageable. The reason for this is that most of the work concerns research, for which agility and creating a proof-of-concept are more important than long term stability. Hence, pruning of code and complete classes happens more often. Usually a yearly clean-up session of essential code is considered to keep a working base and release a new stable version of DEMKit, which uses proven and (nearly) published work. Due to the nature of research, most documentation is also provided in academic output (papers, journals and Master and PhD dissertations).

6.1 DEMKit framework

The DEMKit framework has a couple of coding styles:

- Use tabs instead of spaces
- CamelCase usage for function and variable names, start with small letter
- Clear variable names
- Classes start with capital letter
- Filenames start with small letter, also use CamelCase

Development takes place in the **development** branch, whereas the **master** branch contains the last stable(-ish) version of DEMKit. New features are developed both in separate branches as the **develop** branch. If development takes place in the development branch, make sure that new additions do not cause a broken simulator. Also, add warning statements (e.g. in the **startup** function of a component) that certain features are experimental.

Logging of data also follows a standard format for clarity. Please use metric numbers and follow SI conventions (except for energy, Wh is preferred). Abstain from using Kilo's, Mega's, Milli's, etc. to make calculations using the data more easily. Grafana already adds the prefixes in visualization as well.

Please follow this naming convention:

Unit-type[.optional.number.of.specifiers][.c.commodity][.complex-part]

Examples:

Wh-energy-consumption.c.power

W.power.imported.c.power.real

W.power.imported.c.power.imag

C.temperature.indoor

6.2 Scenarios

Scenarios (in the workspace folder) are not a part of maintained code and offer full flexibility to the user to create scenarios as they please. The only exception is the read-only example file, which is maintained. The example also the scenario that provides information about the parameters that can be set.